Accurate and General Hardware Performance Counting

Project Report by
Lien Zhu

In Partial Fulfillment of the Requirements for the
Degree of
Master of Engineering in Computer Science and Engineering

MCKELVEY SCHOOL OF ENGINEERING AT WASHINGTON UNIVERSITY
St. Louis, Missouri

2023
Defended April 10, 2023

© 2023

Lien Zhu
ORCID: 0009-0001-4496-0033

Some rights reserved. This report is available through the Washington University in St. Louis Open Scholarship repository

To my mother, for dedicating her life to her sons.

ACKNOWLEDGEMENTS

I am eternally grateful to Professor Ning Zhang for granting me the opportunity to work as part of the Computer Security and Privacy Lab, which became paramount to my rapid development as an engineering student. I would like to thank Professor Patrick Crowley and Professor Sanjoy Baruah for graciously agreeing to sit on my defense committee.

Working closely with Ao Li was an experience with no substitute. I am fortunate to have worked with such a skilled and wise mentor and friend early in my engineering career. I enjoy our conversations about scholarship, cinema, and philosophy.

Likewise, I am lucky to have met a top-notch educator, collaborator, and friend in Marion Sudvarg. I am also thankful for Tomson Li and his help in studying operating systems.

I am touched to have found friends in Han Liu, Yujie Wang, and Yuhao Wu who are always available to provide guidance.

ABSTRACT

Users of modern processors typically have a set of special-purpose registers at their disposal for performance profiling. These Hardware Performance Counters (HPC) organized into a Performance Monitoring Unit (PMU) tally one micro-architectural event at a time with the original intent of performance analysis. The PMU has since been implicated in real-time system dynamic resource allocation, debugging, and malware detection. Many critical applications rely on accurate HPC data, yet they are prone to error. One significant source of error stems from the fact that meaningful profile generation often requires tracing many events, using the few HPCs offered by the hardware. This necessitates multiplexing, where events are time-shared across the counters. The values are then scaled up to the entire profiling duration. This extrapolation introduces measurement error and forms the basis of the event scheduling problem. Prior works have used *post-hoc* machine learning techniques to clean data and have leveraged statistical relationships between events in novel event scheduler implementations to improve accuracy. However, these methods lack versatility necessary to apply to all applications. This work presents the development of a suite of tools successful in demonstrating multiplexing effects and noise measurement, as part of an evaluation of a larger accomplishment *Tintin*, an accurate and versatile performance monitoring system that mitigates these errors as well as those from architecture and hardware effects. *Tintin* outperforms existing methods with versatility across a diverse set of use cases and applications.

Chapter 1

AN INTRODUCTION TO HARDWARE PERFORMANCE COUNTING

Users of modern processors typically have a set of special-purpose registers at their disposal for performance profiling. These Hardware Performance Counters (HPC) organized into a Performance Monitoring Unit (PMU) tally one micro-architectural event at a time with the original intent of performance analysis for debugging and tuning. The PMU has since been implicated in real-time system dynamic resource allocation [14], malware detection [28] and large-scale computation systems. Google uses HPCs for performance isolation in its cloud compute servers [34] and to monitor its data centers [26] where efficiency improvements of just 1% can mean savings of millions of dollars.

Many critical applications rely on accurate and precise HPC data, particularly those used for vulnerability research and security monitoring [9], where successful detection of a compromised system often relies on minuscule deviations in event counts. Yet, HPC data is prone to error. One significant source of error stems from the fact that meaningful profile generation often requires tracing many events, yet hardware often only offers a few HPCs. For instance, the Raspberry Pi Model 3B+ only contains 6 HPCs in its PMUs [4] while hundreds of events may have to be counted [20] for some profiling solution. This necessitates multiplexing, where events are time-shared across the counters. The values are then scaled up to the entire profiling duration. This extrapolation introduces measurement error and forms the basis of the event scheduling problem.

Prior works have used *post-hoc* machine learning techniques to clean data and have leveraged statistical relationships between events in novel event scheduler implementations to improve accuracy. CounterMiner [18] applies data mining and machine learning techniques on data that has been gathered from HPCs, after all data has been collected from some program's execution. BayesPerf [5] asserts that prior knowledge of micro-architectural event correlations can be leveraged to schedule event groups that permit Bayesian inference to reduce uncertainty in measurements. However, these methods lack versatility needed to apply generally to various systems and use cases. CounterMiner is effective for profiling but unsuited for online tasks

such as dynamic resource allocation [9]. BayesPerf comes with significant hardware overhead and can fail to produce a schedule that out-performs the vanilla event scheduler if there are insufficient Bayesian relationships between events.

This work presents the development of a suite of tools crafted with the Python and shell scripting languages that demonstrate multiplexing effects and noise measurement. These achievements are part of an evaluation of a larger accomplishment *Tintin*, an accurate and versatile performance monitoring system that reduces error from multiplexing as well as those from architecture and hardware effects.

Briefly, *Tintin* draws from existing methods to create a general three-component system for performance monitoring. *Tintin-Miner* uses sensitivity analysis to prune events that are not insightful into a task's execution behavior. Informed by *Tintin-Miner*, *Tintin-Scheduler* leverages elastic scheduling to solve the event scheduling problem. These two components enable *Tintin-Monitor* to present an identical interface to the Linux perf system while offering more flexible profiling with low overhead and intrusion. *Tintin* aims to outperform existing methods with versatility across a diverse set of use cases and applications from profiling on a Raspberry Pi 3B+ [1] to resource management using the FIRM framework [25] on an Intel Xeon server-grade machine [16].

For the past academic year, I have been working with Ao Li to *systematize* the understanding of errors in HPC based techniques and *reduce* uncertainty in profiling methods that are built on top of the Linux perf subsystem with our efforts to build *Tintin*. This project report will describe what I learned from Ao Li during my transition into computer science as we worked on *Tintin* and the individual contributions I made as a master's student in the Computer Security and Privacy Lab directed by Professor Ning Zhang.

Chapter 2

PERFORMANCE MONITORING AND ITS SUPPORTING INFRASTRUCTURE IN THE LINUX KERNEL

2.1 The Linux `perf_event` Subsystem

Each core of modern processors contains a Performance Monitoring Unit (PMU) which provides an abstraction of a collection of special purpose registers (HPC) that are able to count micro-architectural events, such as cache misses, instructions retired, and branches for example, that occur during some process context or globally.

Linux offers control of the PMU through the Linux perf_event subsystem. The subsystem includes the `perf_event_open()` system call. This interface allows users to configure the PMU to count system-specific events at the process context level. Additionally, the Linux *perf* user space program is included with the subsystem. The *perf* tool [24] uses the `perf_event_open()` syscall in its implementation of a command line interface that allows users to interact with the kernel to profile workload applications and or system-wide behavior, see data in real-time, and generate reports with visualization tools.

2.2 Error Sources

Reading Techniques [31]

Linux `perf_event` supports two methods of reading from counters: polling and sampling.

Linux `perf_event` by default polls or counts events at regular intervals, storing event counts in an internal data structure. This buffer can be read at any time using `read()` on the file descriptor returned by the initial call to `perf_event_open()`. Polling with its one-counter-one-event scheme [18] is often impractical as many events usually must be counted to understand execution behavior [20, 5].

Events can also be sampled. Each HPC has a maximum value, `MAX`, that upon overflow generates a performance monitor interrupt (PMI). When the PMU is configured to sample event counts, the HPC for some selected monitor event A is set to `MAX-P`, where `P` is the desired threshold. Once `P` occurrences of A are recorded, the PMI is triggered and all event counters are read. Sampling can also be conducted when a specified instruction in the code is reached. Both schemes of sampling are suscep-

tible to sampling error, where the measurement taken is not representative of the entire program execution.

Multiplexing and the Vanilla Linux Event Scheduler

Systems support far more events than there are available counters. For instance, running `sudo perf list` on my Raspberry Pi 3B+ lists 7 hardware events and 15 cache specific events. Just these two categories already exceed the 6 counters offered by the hardware [4] by almost 4 times. This necessitates *multiplexing*, or time-sharing of events onto the counters. When the number of events of interest exceed the hardware's capacity, events contend for time on the PMU. When scheduled on the PMU, the event is eligible for counting. Once program execution or profiling has completed, each event count is scaled from the fraction of time the event was scheduled on the PMU to the entire profiling duration.

Extrapolation incurs measurement error due to non-uniform execution profiles [33, 14]. In essence, many programs do not have uniform execution where the rate of any particular event's occurrence is constant. Often, programs run in phases. The start of a process's execution may have far more I/O events than later phases of execution, for example. Scheduling I/O related events early would lead to extrapolation producing an overestimate while scheduling I/O events later in the process's execution would underestimate the true value.

Multiplexing motivates the event-scheduling problem, the assignment of events to counters. Additionally, hardware constraints place limitations on event placements to counters. The Operating System (OS) is responsible for scheduling events onto the PMU through the API provided by `perf_event`. The primitive unit of event scheduling is the event group [10], composed of a leader event that defines the attributes of events that are allowed to join the group and be scheduled together. Groups can be *pinned* [27] to the PMU and will not be multiplexed off once scheduled for some process context, though event groups can also be globally pinned. A group that is not *pinned* is *flexible*.

Linux `perf_event` iterates through a rotating list of event groups and attempts to assign each event in a group to a counter, complying with any constraints on the types of event a counter can support. All events in a group must be scheduled otherwise none are scheduled. The Linux `hrtimer` infrastructure is used to set the event scheduling interval.

Banerjee et al. state that the sampling technique enables multiplexing [5], however I

believe this distinction requires further elaboration. Multiplexing is only relevant to event scheduling. Once a group is scheduled, the PMU configuration will determine whether the events are polled for the entirety of the scheduled time, or merely sampled within the scheduled time.

The vanilla Linux `perf_event` scheduling algorithm [27] can result in starvation of events. Consider the case where there are 6 counters and two groups to be scheduled, *flexible* group `A` composed of 4 events and *pinned* group `B` composed of 3 events. In the first iteration of the scheduler, group `A` would be scheduled and counted while `B` would wait for the next iteration. The second iteration, `B` would be scheduled and `A` would wait. However, upon the third iteration, group `A` does not get multiplexed off the PMU because of its *pinned* property. This would lead to `B` being starved of time on the PMU. The drastically diminished fraction of time that `B` was scheduled would exacerbate extrapolation error.

Context Switches and Contamination

The PMU operates asynchronously from the CPU and operating system. Improper saving and restoration of counter values upon context switches has been documented to result in contamination [9] of event counts across processes. In essence, the event counts of one process are attributed to another process.

Overcounting and Hyper-thread Corruption

Certain architectures have been demonstrated to overcount [30] some events when executing some instructions. The retired instruction count has been demonstrated to be susceptible to overcounting during execution of some instructions (`fldcw, fldenv, emms`) on Pentium D processors. Additionally, many x86-64 machines increment all event counts upon interrupt receipt. This effect is attributed to architectural design.

Another source of architecture dependent error is corruption between threads of the same processor core. Certain Intel processors capable of the company's Hyper-Threading technology are prone to this corruption, where counts from one thread's counters leak into one of its sister's counters, resulting in inaccurate counts [10].

Chapter 3

TINTIN OVERVIEW

Briefly, I will describe *Tintin* in order to place my individual contribution in a larger scope when my individual work is discussed in the following chapter.

What to measure? *Tintin-Miner*

The key insight into *Tintin-Miner* is that pruning out events that have insignificant change during a program's execution would reduce the multiplexing load, giving more significant events increased time on the PMU and more accurate profiles. *Tintin-Miner* uses sensitivity analysis to quantify, rank, and prune events based on their importance with respect to desired performance indicators such as execution time, cycles per instruction, or throughput. This can inform a smaller subset of events for future profiling, reducing the effects of multiplexing. Miner is also able to determine counts with anomalies to identify non-deterministic execution as possible signs of malware infestation.

When to measure? *Tintin-Scheduler*

Tintin-Scheduler provides an abstraction over which scheduling policies can be constructed to fit desired profiling goals. It uses elastic scheduling to set each event's utilization of a counter according to the ranking informed by *Tintin-Miner*. It can dynamically update the schedule based on online measurements of count variance, such that events with more variance are allowed more time to occupy a counter in hopes of reducing the variance. *Tintin-Scheduler* supports constrained assignment where special-purpose HPCs can only monitor a subset of events. It also supports an event group abstraction (similarly to Linux perf [29] where events are co-scheduled to identify relationships as is done in BayesPerf [5].

How to measure? *Tintin-Monitor*

Finally, *Tintin-Monitor* is a tool that presents a similar interface to Linux `perf` that is tightly coupled with *Tintin-Miner* and *Tintin-Scheduler*. The three components work together with *Tintin-Miner* executing *Tintin-Monitor* first to get some data and *Tintin-Monitor* then executing *Tintin-Scheduler* to formulate an event schedule. *Tintin-Monitor* comes with low overhead and intrusion to the profiled application.

Chapter 4

INDIVIDUAL ACCOMPLISHMENTS: STUDYING COMPUTER SCIENCE, READING LITERATURE, AND REPRODUCING ERRORS

Code will be released after publication decision.

4.1 Literature Review and Tooling Up

Working with Ao, I learned that to start a paper, we must first understand the current landscape of the field to learn where we can find some opportunity to make improvements. I began my individual contribution by conducting a literature review of around 20 papers to gather more information about hardware performance counting, which is coarsely organized in Table 4.1.

Table 4.1: Hardware Performance Counters in Literature by Focus

Security and Debugging	Resource Allocation	General/ Error Correction
Wang and Karri [28]	Gottschall et al [15]	Banerjee et al [5]
Das et al [9]	Ren et al [26]	Moseley et al [20]
Bechtel and Yun [6]	Gifford et al [14]	Ammons et al [3]
Mergendahl et al [19]	Zhang et al [34]	Weaver et al [30]
Geier et al [12]	Yun et al [32]	Zellweger et al [33]
Pan [23]	Gifford et al [13]	Neill et al [22]
	Courtaud et al [8]	Mytkowicz et al [21]
	Farrukh et al [11]	Lv et al [18]
	Ali and Yun [2]	

Because my undergraduate training was not in engineering, I sought out resources to become useful. This included reading *Computer Systems, a Programmer's Perspective* by Randal E. Bryant and David R. O'Hallaron to better understand computers and working through operating systems course materials on a Raspberry Pi.

This process proved fruitful as it helped make learning from Ao faster and made it possible for me to contribute to the larger project in our discussions and experiments. I was able to develop programs at every level of the system stack. I will start by describing my main contribution and learning in user space then work my way to lower levels.

4.2 Preliminary and Evaluation Studies Using Python and Shell

We set out to reproduce errors described in the literature in order to motivate our research. This required measuring two effects.

Multiplexing Effects

First, we wanted to measure the effects of *multiplexing*. The idea behind this study is to run `perf` on a workload many times with a different number of events counted at the same time. I developed a tool suite composed of Python programs and Bash Shell scripts that accomplishes invocation of `perf` on the stress next generation [17] and the SPEC CPU2017 [7] benchmark tools as workloads, as well as processing and visualization of data. All together, there are about 2K LOC spread about approximately 30 Python modules. Each program was built with modularity in mind and uses object-oriented design to achieve this requirement. Additionally, there is robust support for command line arguments that keep the program flexible to the needs of the user. These programs will be released as part of the open source *Tintin* project for the community to be able to replicate our results and explore the effects on different architectures.

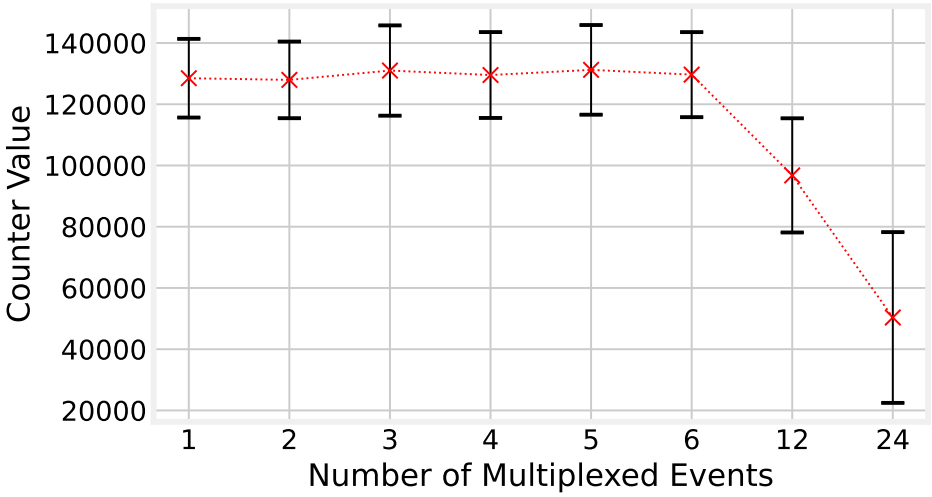

Figure 4.1: The effects of multiplexing on RPi 3B+ tracing stress-ng

Each PMU on the Raspberry Pi 3B+ has 6 HPCs [4]. To demonstrate the effects of multiplexing, I ran `perf_5.10` on `stress-ng` with 1 - 6, 12, and 24 events counted at a time. I pre-selected the event `LLC-load-misses` and plotted the counter value and one standard deviation above and below from 500 trials for each run. Figure 4.1 shows that when the multiplexing is not needed, when the number of events of interest

is less than or equal to the number of HPCs, event counts are stable. Increasing the number of traced events induces multiplexing and the variance increases as expected.

Phase Sensitivity and Interval Tracking

For the development of *Tintin-Miner*, we conducted preliminary sensitivity analysis that will be the foundation for the final product. This required obtaining counter values at intervals within the workload execution duration, rather than just the total accumulated counts at the end. Because of the modularity of the product I produced for the multiplexing studies, I only had to make minor adjustments in order to have code we could use for the sensitivity studies.

Granularity or Scope Effects

The second major individual work produced is a system for comparing error due to measurement granularity. We reasoned that using the `perf` tool at a system level would have greater error than invoking `perf` on a workload task. Staying within the capabilities of the Linux `perf_event` interface, instrumenting a workload with `perf_event_open()` would have the smallest variance. I was tasked with designing a system to demonstrate this reasoning.

First, I will describe the process of instrumenting `perf_event_open()` into SPEC CPU2017 to get the *instrumented* data. I quickly learned that benchmarks are not designed to be modified by the end user. After a painstaking process and lots of consultation with Ao, I finally figured out where to put the syscalls and reads, and where the output was going. I instrumented the 502.gcc_r benchmark with code that calls `perf_event_open()` and reads from the file descriptors to get the data. Then, I made a Bash script to run the workload many times, with Python graphing `LLC-load-misses` counts.

Then, I had to figure out how to get the *system-wide* and *task-level* data. The *task-level* data was easy as the Python programs used for the multiplexing studies could be used to run `perf` on 502.gcc_r with `LLC-load-misses` as the sole event of interest. However, the *system-wide* experiments required some engineering. At first, I tried using Python but setting up the script to run a sub-process with the appropriate environment proved too difficult. I considered Bash scripting. This was the best solution for this problem. The critical snippets of this script first invokes `perf stat -e LLC-load-misses &` to track the event in the background, then immediately runs SPEC CPU2017's 502.gcc_r benchmark. Once the CPU2017 workload completes, the shell script then uses `pkill perf` to terminate the `perf`

process and obtain the *system-wide* data.

I then slightly modified the Python scripts I wrote previously that handled data visualization to graph the *system-wide*, *task-level*, and *instrumented* data side by side for comparison. Additionally, I set up automation tools so the suite of tools could be run repeatedly to accelerate experiment execution.

4.3 Toy Experiments and Lower Level Learning

As we experimented with Linux `perf_event`, I picked up several skills that I can take into industry and future academic ventures.

Writing loadable kernel modules

Because the `hrtimer` subsystem is used for event scheduling intervals in the PMU, any inaccuracy in this infrastructure would affect event scheduling timing. I learned how to write loadable kernel modules to measure `hrtimer` inaccuracy. Essentially, the module prints time readings using `hrtimer` upon loading until the module is unloaded. This toy experiment let me learn about kernel modules.

Using Make and creating a kernel build system

One of the toy experiments we conducted required me to set up a build system for custom changes the Linux kernel. We were interested in timing `perf_event_open()`. I first had to figure out where in the Linux source events are configured for counting on the PMU, and where they are removed from PMU. I narrowed it down from `perf_event_open()` to drivers/perf/arm_pmu.c on the Raspberry Pi. I used the `ktime` infrastructure to place a starting time recording when `armpmu_add()` is called and an ending time recording when `armpmu_del()` is called. I stored these 1000 of these counts in an internal buffer when running the `perf` tool, which was then printed to the kernel log when 1000 times were recorded. This demonstrated rough estimates of jitters.

This process forced me to learn how to set up a build system using several Bash scripts that invoke Make to cross-compile our changes to the Linux kernel. The system is still being used by the team, even if the experiment was just for rough estimation.

4.4 Miscellaneous and Lasting Lessons

During the course of this project, I learned how to use Vim, which is now my favorite editor. Working on a remote machine gave me the opportunity to learn

tmux and several network technologies including `ssh`, `scp`, and `sftp`. I learned what an operating system is, some assembly, how program compilation works, instruction pipe-lining, virtualization, the cache hierarchy and a little bit of computer architecture, and what abstraction really means. From the systems concepts I learned, I feel more confident working on computers as I now understand the underlying mechanisms behind their operation.

Overall, I learned more in this year than I had in four years of undergrad. I learned how to find and address limitations in my knowledge, communicate my ideas across different levels of technical experience, and make decisions in allocating human capital which will be critical for my later career. I would not have learned as much as I did this year if I was not in Professor Zhang's lab. Having access to a wonderful professor and brilliant PhD students was invaluable. I did not expect my decision to join the CSPL to be so crucial to my development as an engineer, which taught me the greatest lesson I learned this year: the choice is truly more important than the effort.

REFERENCES

[1] https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/. Accessed on March 25, 2023.

[2] Waqar Ali and Heechul Yun. *RT-Gang: Real-Time Gang Scheduling Framework for Safety-Critical Systems*. 2019. arXiv: 1903.00999 [cs.DC].

[3] Glenn Ammons, Thomas Ball, and James R. Larus. "Exploiting Hardware Performance Counters with Flow and Context Sensitive Profiling". In: *Proceedings of the ACM SIGPLAN 1997 Conference on Programming Language Design and Implementation*. PLDI '97. Las Vegas, Nevada, USA: Association for Computing Machinery, 1997, pp. 85–96. ISBN: 0897919076. DOI: 10.1145/258915.258924. URL: https://doi.org/10.1145/258915.258924.

[4] Arm. *About the PMU*. https://developer.arm.com/documentation/ddi0500/j/Performance-Monitor-Unit/About-the-PMU?lang=en. Accessed March 24, 2023. 2023.

[5] Subho S. Banerjee et al. "BayesPerf: Minimizing Performance Monitoring Errors Using Bayesian Statistics". In: *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS '21. Virtual, USA: Association for Computing Machinery, 2021, pp. 832–844. ISBN: 9781450383172. DOI: 10.1145/3445814.3446739. URL: https://doi.org/10.1145/3445814.3446739.

[6] M. Bechtel and H. Yun. "Denial-of-Service Attacks on Shared Cache in Multicore: Analysis and Prevention". In: *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. Los Alamitos, CA, USA: IEEE Computer Society, Apr. 2019, pp. 357–367. DOI: 10.1109/RTAS.2019.00037. URL: https://doi.ieeecomputersociety.org/10.1109/RTAS.2019.00037.

[7] Standard Performance Evaluation Corporation. *SPEC CPU2017*. https://www.spec.org/cpu2017/. [Online; accessed 24 March 2023]. 2017.

[8] Cédric Courtaud et al. "Improving Prediction Accuracy of Memory Interferences for Multicore Platforms". In: *2019 IEEE Real-Time Systems Symposium (RTSS)*. 2019, pp. 246–259. DOI: 10.1109/RTSS46320.2019.00031.

[9] Sanjeev Das et al. "SoK: The Challenges, Pitfalls, and Perils of Using Hardware Performance Counters for Security". In: *2019 IEEE Symposium on Security and Privacy (SP)*. 2019, pp. 20–38. DOI: 10.1109/SP.2019.00021.

[10] Maria Dimakopoulou et al. "Reliable and Efficient Performance Monitoring in Linux". In: *SC '16: Proceedings of the International Conference for High*

Performance Computing, Networking, Storage and Analysis. 2016, pp. 396–408. DOI: 10.1109/SC.2016.33.

[11] Anam Farrukh and Richard West. "FLYOS: Integrated Modular Avionics for Autonomous Multicopters". In: *2022 IEEE 28th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2022, pp. 68–81. DOI: 10.1109/RTAS54340.2022.00014.

[12] Martin Geier et al. "Debugging FPGA-accelerated Real-time Systems". In: *2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2020, pp. 350–363. DOI: 10.1109/RTAS48715.2020.00010.

[13] Robert Gifford and Linh Thi Xuan Phan. "Multi-mode on Multi-core: Making the best of both worlds with Omni". In: *2022 IEEE Real-Time Systems Symposium (RTSS)*. 2022, pp. 118–131. DOI: 10.1109/RTSS55097.2022.00020.

[14] Robert Gifford et al. "DNA: Dynamic Resource Allocation for Soft Real-Time Multicore Systems". In: *2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2021, pp. 196–209. DOI: 10.1109/RTAS52030.2021.00024.

[15] Björn Gottschall, Lieven Eeckhout, and Magnus Jahre. "TIP: Time-Proportional Instruction Profiling". In: *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. MICRO '21. Virtual Event, Greece: Association for Computing Machinery, 2021, pp. 15–27. ISBN: 9781450385572. DOI: 10.1145/3466752.3480058. URL: https://doi.org/10.1145/3466752.3480058.

[16] Intel Corporation. *Intel® Xeon® Gold 6130 Processor (22M Cache, 2.10 GHz)*. https://ark.intel.com/content/www/us/en/ark/products/120492/intel-xeon-gold-6130-processor-22m-cache-2-10-ghz.html. Accessed on March 25, 2023. Mar. 2018.

[17] Colin Ian King. *stress-ng*. https://github.com/ColinIanKing/stress-ng. Accessed on 24 March 2023. 2014.

[18] Yirong Lv et al. "CounterMiner: Mining Big Performance Data from Hardware Counters". In: *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 2018, pp. 613–626. DOI: 10.1109/MICRO.2018.00056.

[19] Samuel Mergendahl et al. "The Thundering Herd: Amplifying Kernel Interference to Attack Response Times". In: *2022 IEEE 28th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2022, pp. 95–107. DOI: 10.1109/RTAS54340.2022.00016.

[20] Tipp Moseley, Neil Vachharajani, and William Jalby. "Hardware Performance Monitoring for the Rest of Us: A Position and Survey". In: *8th Network and Parallel Computing (NPC)*. Ed. by Erik Altman and Weisong Shi. Vol. LNCS-6985. Network and Parallel Computing. Part 8: Session 8: Microarchitecture.

Changsha,, China: Springer, Oct. 2011, pp. 293–312. DOI: 10.1007/978-3-642-24403-2_23. URL: https://hal.inria.fr/hal-01593009.

[21] Todd Mytkowicz et al. "Time Interpolation: So Many Metrics, So Few Registers". In: *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*. 2007, pp. 286–300. DOI: 10.1109/MICRO.2007.27.

[22] Richard Neill, Andi Drebes, and Antoniu Pop. "Fuse: Accurate Multiplexing of Hardware Performance Counters Across Executions". In: *ACM Trans. Archit. Code Optim.* 14.4 (Dec. 2017). ISSN: 1544-3566. DOI: 10.1145/3148054. URL: https://doi.org/10.1145/3148054.

[23] Runyu Pan and Gabriel Parmer. "SBIs: Application Access to Safe, Baremetal Interrupt Latencies". In: *2022 IEEE 28th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2022, pp. 82–94. DOI: 10.1109/RTAS54340.2022.00015.

[24] perf.wiki.kernel.org. *Linux perf wiki*. https://perf.wiki.kernel.org/index.php/Main_Page. Accessed March 24, 2023. 2023.

[25] Haoran Qiu et al. "FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices". In: *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 805–825. ISBN: 978-1-939133-19-9. URL: https://www.usenix.org/conference/osdi20/presentation/qiu.

[26] Gang Ren et al. "Google-Wide Profiling: A Continuous Profiling Infrastructure for Data Centers". In: *IEEE Micro* (2010), pp. 65–79. URL: http://www.computer.org/portal/web/csdl/doi/10.1109/MM.2010.68.

[27] Linux Torvalds and Contributors. *Linux Kernel*. [Source code]. 1991. URL: https://github.com/torvalds/linux.

[28] Xueyang Wang and Ramesh Karri. "NumChecker: Detecting kernel control-flow modifying rootkits by using Hardware Performance Counters". In: *2013 50th ACM/EDAC/IEEE Design Automation Conference (DAC)*. 2013, pp. 1–7.

[29] Vincent M Weaver. "Linux perf_event features and overhead". In: *The 2nd international workshop on performance analysis of workload optimized systems, FastPath*. Vol. 13. Austin, TX: -, 2013, p. 5.

[30] Vincent M. Weaver, Dan Terpstra, and Shirley Moore. "Non-determinism and overcount on modern hardware performance counter implementations". In: *2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 2013, pp. 215–224. DOI: 10.1109/ISPASS.2013.6557172.

[31] Hao Xu et al. "Can We Trust Profiling Results? Understanding and Fixing the Inaccuracy in Modern Profilers". In: *Proceedings of the ACM International Conference on Supercomputing*. ICS '19. Phoenix, Arizona: Association for Computing Machinery, 2019, pp. 284–295. ISBN: 9781450360791. DOI: 10.1145/3330345.3330371. URL: https://doi.org/10.1145/3330345.3330371.

[32] Heechul Yun et al. "MemGuard: Memory bandwidth reservation system for efficient performance isolation in multi-core platforms". In: *2013 IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2013, pp. 55–64. DOI: 10.1109/RTAS.2013.6531079.

[33] Gerd Zellweger, Denny Lin, and Timothy Roscoe. "So Many Performance Events, so Little Time". In: *Proceedings of the 7th ACM SIGOPS Asia-Pacific Workshop on Systems*. APSys '16. Hong Kong, Hong Kong: Association for Computing Machinery, 2016. ISBN: 9781450342650. DOI: 10.1145/2967360.2967375. URL: https://doi.org/10.1145/2967360.2967375.

[34] Xiao Zhang et al. "CPI^2: CPU performance isolation for shared compute clusters". In: *SIGOPS European Conference on Computer Systems (EuroSys)*. Prague, Czech Republic, 2013, pp. 379–391. URL: http://eurosys2013.tudos.org/wp-content/uploads/2013/paper/Zhang%5C_2.pdf.